

Resumen del repositorio ad-astra-retrieval

CODEFEST AD ASTRA 2026 · Etapa 1 — Base de Conocimiento Vectorial · Preparado para revisión de equipo

1. Qué construyó

Un sistema de **recuperación densa multilingüe (RAG sin generación)**: indexa el corpus oficial de ADL (1826 documentos en español/inglés/portugués, sobre los tres fenómenos del reto) en una base vectorial FAISS y, dadas 50 consultas en lenguaje natural, devuelve para cada una los 3 documentos y los 10 fragmentos más relevantes, sin usar en ningún punto un modelo generativo.

Estado declarado por el propio autor (README): entrega completa — 1 821 de 1 826 documentos indexados, 90 613 fragmentos, `resultados.jsonl` con 50 líneas sin errores de esquema, grafo de conocimiento integrado, 65 pruebas automatizadas.

2. Pipeline de extremo a extremo

Extracción → limpieza/normalización → chunking (dos niveles) → codificación con BGE-M3 (denso + disperso) → índice FAISS (IndexFlatIP) → fusión ponderada RRF (con el grafo como ranking adicional) → reranking opcional con cross-encoder → agregación chunk→documento → validación estricta de esquema → `resultados.jsonl`.

- **Extracción** por formato: PDF con Docling (respaldo PyMuPDF), HTML con trafilatura, JSON por selección explícita de campos, CSV/XLSX fila→ pares columna:valor, imágenes con OCR (EasyOCR), mapas .pbf (vector tiles) recorriendo capas y atributos.
- **Limpieza**: normalización NFC, eliminación de líneas de índice y boilerplate repetido, re-unión de palabras cortadas por guion.
- **Chunking en dos niveles**: Nivel 1 agrupa oraciones completas hasta ~256-384 tokens con solapamiento de una oración (unidad que se indexa); Nivel 2 re-divide al construir la salida para respetar el límite de 250 palabras, siempre en frontera de oración.
- **Codificación**: BGE-M3 (denso 1024d + señal léxica dispersa) sobre cada fragmento.
- **Indexación**: FAISS IndexFlatIP con vectores normalizados (coseno exacto) + `metadata.jsonl` alineado 1:1 por línea.
- **Recuperación**: mismo encoder para la consulta, fusión RRF ponderada de denso + disperso + grafo (peso 0,3), reranking opcional con cross-encoder, agregación por documento (max pooling).

3. Decisiones técnicas clave y su justificación

Decisión	Elegido	Por qué (referencia a la spec)
Encoder	BAAI/bge-m3 (denso+disperso)	Cross-lingual nativo ES/EN/PT (§4.3), 8192 tokens, MIT, medido 0.990 vs 0.966 de un multilingüe pequeño.
Índice FAISS	IndexFlatIP normalizado	Coseno exacto (§5.2); el volumen del corpus no justifica sacrificar exactitud por velocidad, que no se puntúa.
Fusión multi-señal	RRF ponderada (k0=60)	§8.4, ecuación 7; extendida con pesos por ranking para que el grafo (0,3) no desplace aciertos del denso (peso 1,0).
Reranking	Cross-encoder bge-reranker-v2-m3 (toggle)	§8.3 prohíbe decoders; el equipo interpreta que un cross-encoder es familia BERT y consultó al jurado, que respondió que sí está permitido.
Agregación a documento	max pooling	§8.6; medido mejor F1@3 que suma o media ponderada en dos corpus distintos.
Grafo de conocimiento (bonus)	Implementado: GLiNER (NER) + co-ocurrencia, NetworkX → GraphML	§7; integrado a la recuperación como ranking adicional en la fusión (§8.5), porque el propio organizador aclaró que el bono solo puntúa si está conectado al retrieval.

4. Cómo responde a los requisitos del PDF de la organización

- **Chunking y completitud lingüística (§3.2-3.3):** chunking por oraciones con solapamiento, en vez de tamaño fijo, para no violar el requisito de no cortar oraciones. El equipo detectó y corrigió un defecto propio en el que 35/500 fragmentos sí violaban esta regla (bitácora, hallazgo C18).
- **Metadata obligatoria (Tabla 1, §3.4):** los ocho campos exigidos están presentes en `metadata.jsonl`, más campos extra permitidos por la spec.
- **Prohibición de decoders (§8.3):** todos los modelos usados son encoder (BGE-M3, GLiNER, bge-reranker) según el propio equipo. El reranking cross-encoder es el punto más discutible; el equipo documenta que consultó formalmente al jurado y que hay un interruptor para desactivarlo si la interpretación fuera restrictiva.
- **Esquema de salida (§9.3):** validador estricto antes de escribir cada línea (3 documentos, 10 fragmentos, ≤ 250 palabras); `resultados.jsonl` tiene exactamente 50 líneas.
- **Persistencia FAISS + metadata (§5.4, §1.4):** `index.faiss` vía `faiss.write_index()` y `metadata.jsonl` alineado por orden de inserción, como exige la spec.
- **Grafo bonus (§7):** construido y fusionado a la recuperación, no solo exportado — condición que el propio organizador aclaró como necesaria para que puntúe.

Punto de atención — emparejamiento a nivel documento (§10.2.1): el enunciado dice que el emparejamiento se hace por el campo `fuelle`, pero el propio equipo reporta en su documentación que ADL corrigió esto por correo/FAQ: en realidad se empareja por `doc_id` del inventario oficial. El repo ya usaba `doc_id` desde el principio. **Esto no está en el PDF que se nos entregó a nosotros** — es una aclaración externa que el equipo recibió; conviene confirmar con la organización antes de dar esto por cerrado.

5. Estado de la entrega y limitaciones declaradas

Completo:

- `resultados.jsonl` (50 líneas, 0 errores de esquema).
- `generador.py` (reproducible: dos corridas dan el mismo MD5).
- `informe_tecnico.pdf` (documento técnico de diseño, dentro del límite de 8 páginas).
- Grafo de conocimiento (bonus) integrado a la recuperación.
- 65 pruebas automatizadas (pytest).

Atajo a revisar — base vectorial no está en el repositorio:

`index.faiss` (354 MB) y `metadata.jsonl` (190 MB) superan el límite de 100 MB de GitHub y no están en entrega/localmente: se distribuyen aparte como release de GitHub y hay que descomprimirlos manualmente. **Confirmar que el mecanismo de entrega del reto acepta esto**, porque el PDF pide el índice y la metadata dentro del directorio único de entrega.

Limitaciones que el propio autor declara:

- Las 50 consultas reales no traen juicios de relevancia (ground truth): ninguna cifra reportada ($NDCG@10 \approx 0.42$, $F1@3 \approx 0.50$) mide el desempeño real en el reto; solo sirvieron para comparar configuraciones entre sí sobre un corpus de prueba propio.
- 5 documentos del inventario oficial no pudieron indexarse (4 fotos sin texto + 1 archivo de 2 bytes).
- El aporte medible del grafo es pequeño: con el peso calibrado (0,3) cambia solo 2 de 50 consultas.
- Riesgo térmico: con los tres modelos en GPU de 8 GB, `generador.py` llegó a 83°C sin pausas (mitigado con la bandera `--pausa`).
- El indicador interno de "coherencia temática" está sesgado (asume que un documento relevante para un fenómeno vive en esa carpeta, lo cual no siempre es cierto) y no debe usarse solo.

Nota: el informe técnico en `docs/informe_tecnico.md` menciona 42 pruebas automatizadas, mientras que el README y la bitácora hablan de 65 — son versiones de distintas fechas del mismo documento; conviene verificar cuál

quedó en el PDF final entregado.

6. Cómo reproducir/ejecutar

- Entorno: Python 3.13, `pip install -r requirements.txt` (instalar torch con CUDA antes que el resto: otras dependencias lo sobrescriben con la build de CPU en silencio).
- Con la base vectorial ya descargada del release de GitHub en `entrega/base_vectorial/`: `python entrega/generador.py --config config.adl.yaml`.
- Desde cero (requiere el corpus de ADL, no incluido en el repo): `extract_corpus.py` → `ocr_scanned.py` → `build_index.py` → `build_graph.py` → `generador.py` → `check_results.py`.
- Verificación: `python -m pytest -q` (65 pruebas) y `python scripts/check_results.py --resultados entrega/resultados.jsonl`.

7. Puntos para que el equipo revise antes de la entrega final

- **Base vectorial fuera del repo:** confirmar si el formulario de entrega del reto acepta un enlace externo (release de GitHub) para `index.faiss` y `metadata.jsonl`, o si deben empaquetarse físicamente en el directorio único.
- **Reranking con cross-encoder:** es la mayor ganancia medida (+17% NDCG@10) pero vive en zona gris del reglamento oficial (§8.3 dice "exclusivamente vectores, puntuaciones y metadata"); el equipo dice haber consultado al jurado por fuera del PDF — pedir la confirmación por escrito si aún no se tiene, para no arriesgar la descalificación de la entrega.
- **Aclaración de doc_id vs fuente:** el repo actúa según una corrección de ADL que no aparece en el documento que tenemos — validar que esa corrección sea oficial y esté vigente para esta edición.
- **Discrepancia de conteo de pruebas** (42 vs 65) entre el borrador del informe técnico y el README/bitácora: revisar cuál cifra quedó en el PDF que se va a entregar.
- **Grafo con aporte marginal:** decidir en equipo si vale la pena mantenerlo activo dado el riesgo de zona gris regulatoria adicional (uso de NER) a cambio de solo 2/50 consultas afectadas.

Fuentes: `CODEFEST_2026-1.pdf` (especificación oficial) y `ad-astra-retrieval/README.md`, `docs/SISTEMA.md`, `docs/BITACORA.md`, `docs/informe_tecnico.md`, `config.adl.yaml`, `entrega/`.